

CERTIFICAÇÃO DATA ENGINEER · INDICIUM

BanVic

Plataforma de ingestão de dados

Infraestrutura como código, pipeline ELT e orquestração para tirar o banco das planilhas

David Couto

O problema

O BanVic decide por dados sem ter os dados no lugar certo.

- 1 Análise manual** Planilhas e apresentações; nenhuma ferramenta de BI em uso.
- 2 Dado preso na origem** ERP, CRM e marketing em sistemas separados, sem ponto único de consulta.
- 3 Risco operacional** A equipe do André repete extrações à mão, com o erro humano que isso carrega.
- 4 Piloto travado** O projeto de crédito do Lucas precisa de base confiável para provar valor.

A POC entrega o caminho automatizado, reprodutível e auditável do ERP até um ambiente único de consulta.

Escopo da entrega

O que está dentro da POC e o que é jornada seguinte.

Dentro

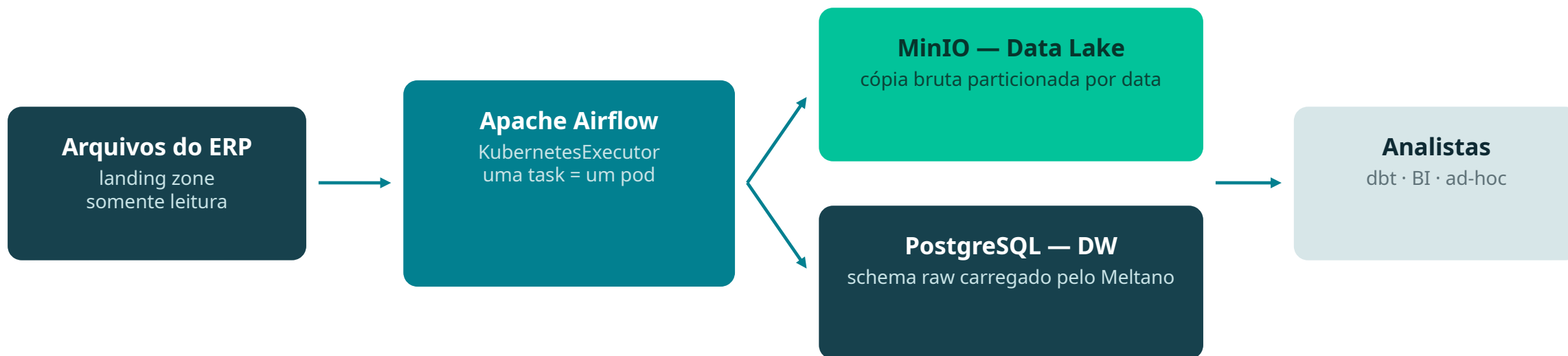
- Infraestrutura como código do ambiente inteiro
- Pipeline de ingestão das 7 tabelas do ERP
- Orquestração, tratamento de falha e idempotência
- Validação de qualidade e auditoria das execuções
- Documentação e demonstração funcional

Etapas seguintes

- Transformação com dbt sobre o schema raw
- Dashboard comercial para a área da Camila
- Ranking de alavancas pedido pela CEO
- Observabilidade com Prometheus e Grafana
- Ingestão incremental quando a origem for o banco

Arquitetura

Cluster Kubernetes local provisionado por Terraform, do arquivo ao analista.



Terraform provisiona namespace, Secrets, volumes, PostgreSQL, MinIO e o release Helm do Airflow. Um comando recria o ambiente do zero.

Stack e por quê

Cada escolha resolve um problema concreto da POC.

kind	Cluster Kubernetes local	Sobe em container, leve, publica as portas do nó direto no host
Terraform	Infraestrutura como código	Um apply recria o ambiente inteiro; estado explícito
Airflow 2.10	Orquestração	KubernetesExecutor: cada task é um pod, sem Celery nem Redis
Meltano	Extract e Load	Padrão Singer, configuração declarativa, MERGE por chave
MinIO	Data Lake	API S3 — mesma interface que a nuvem
PostgreSQL 16	Data Warehouse	Destino analítico simples e conhecido pelo time

Estratégia de ingestão

ELT: carrega primeiro, transforma depois. Dois destinos, propósitos diferentes.

Data Lake

Cópia byte a byte, particionada por data de referência.

`raw/erp/<tabela>/data_referencia=<ds>/`

Registro imutável do que foi recebido. Permite reprocessar qualquer data sem voltar ao ERP.

Data Warehouse

Schema raw carregado pelo Meltano.

`tap-csv → target-postgres`

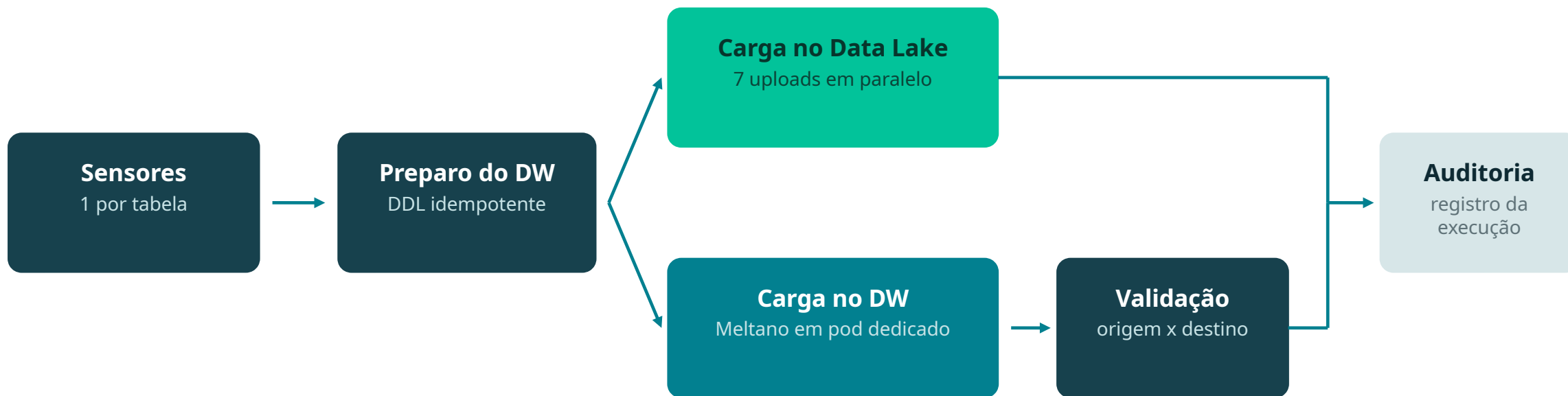
Chave primária declarada por tabela e colunas `_sd*_*` de rastreabilidade de origem.

Idempotência

A partição do Data Lake é sobrescrita e o target-postgres faz MERGE em vez de INSERT. Reprocessar a mesma data não duplica nada — verificado em três execuções seguidas.

A DAG

banvic_erp_ingestao · diária às 04:35 · sem catchup · execução única simultânea



- Sensores garantem que nada comece com fonte incompleta.

- Dois ramos paralelos: não dependem um do outro.

- A validação só roda depois do Meltano — dependência real.

Resiliência

Erro transitório e erro de dado são tratados de formas diferentes.

Pod demora a subir, banco reiniciando

retries=3 com backoff exponencial, teto de 10 min

Arquivo ausente ou vazio

AirflowFailException — falha imediata, sem gastar retry

Contagem divergente, chave nula ou duplicada

Validação por tabela derruba a execução

Task travada

execution_timeout de 30 min (45 min no Meltano)

Pod destruído com o log dentro

Log remoto no MinIO

Falha em produção

on_failure_callback grava status FALHA na auditoria

Pods com falha não são removidos — ficam disponíveis para kubectl logs e describe no post-mortem.

Segurança e segredos

Quem clona o repositório não recebe credencial nenhuma.

- 1 Geradas na hora** `openssl rand` na primeira execução, em arquivo fora do versionamento
- 2 Viram Secret** Terraform declara as variáveis como sensitive e cria Secrets do Kubernetes
- 3 Chegam por ambiente** Pods consomem via `envFrom secretRef` — nunca por arquivo
- 4 Connections injetadas** `AIRFLOW_CONN_*` não ficam no banco nem são digitadas na interface
- 5 Meltano referencia nomes** `meltano.yml` usa `${BANVIC_DW_PASSWORD}`, nunca o valor

Resultado

Sete tabelas do ERP em duas camadas, com contagem conferida a cada execução.

76.206

registros carregados

7 tabelas · 2 camadas
3 execuções com resultado idêntico

agencias	10
clientes	998
colaboradores	100
colaborador_agencia	100
contas	999
propostas_credito	2.000
transacoes	71.999

A conferência é uma task da DAG: compara a contagem do arquivo de origem com a do destino e derruba a execução se divergir.

Próximos passos

O que a POC deixa preparado para a jornada de dados do BanVic.

- 1 Transformação com dbt** Sobre o schema raw: tipagem, padronização e modelo dimensional.
- 2 Dashboard comercial** Transações por cliente, base ativa e sinais de churn para a área da Camila.
- 3 Ranking de alavancas** Com transações, contas, propostas e agências no mesmo lugar, a pergunta da CEO vira consulta.
- 4 Observabilidade** Prometheus e Grafana sobre o exporter StatsD que já está ativo.
- 5 Ingestão incremental** CDC quando a origem deixar de ser arquivo e passar a ser o banco do ERP.

Demonstração

Infraestrutura como código, ingestão idempotente, orquestração com dependências explícitas e nenhum segredo versionado.

`make up`

sobe o ambiente do zero

`make run-dag`

ativa e dispara a DAG

`make verify`

confere os dados no Data Warehouse

`make datalake`

lista os objetos no MinIO

github.com/dwsilva/de_cert_indicium